ASSIGNMENT 3 – TASK 4

Thread Creation and the Thread Life Cycle

Programming in Java | CO3 | 7 Marks | Bloom's Level: K4 – Analyse

1. Problem interpretation

The examination cell has four scanner stations. Each station processes a batch of answer scripts concurrently and reports the number scanned. The implementation demonstrates both ways of defining work: extending Thread and implementing Runnable. The four station batches are independent; the total count is shared and must be updated safely.

2. Thread creation approaches

Extending Thread combines the task and the thread object in one class. It is straightforward for a simple dedicated thread, but the class cannot extend another class. Implementing Runnable separates the task from the thread that executes it. This is generally more flexible because the task can be passed to different Thread instances or executors, and the class remains free to extend another class.

3. Thread life-cycle diagram

```
NEW --start()--> RUNNABLE --chosen by scheduler--> RUNNING*
                          ^                         |
                          |                         | run() completes
                          |                         v
              sleep time expires / lock acquired   TERMINATED
                          ^
                          |
RUNNABLE --sleep()/wait()/join()/lock wait--> BLOCKED / WAITING / TIMED_WAITING

* Java's public Thread.State enum groups ready-to-run and running execution
  under RUNNABLE; RUNNING is often shown separately in conceptual diagrams.
```

A station is NEW after its Thread object is constructed. Calling start() moves it to RUNNABLE. The scheduler chooses when it executes. Thread.sleep() causes TIMED_WAITING; after the delay it becomes eligible to run again. Calling join() on a worker makes the calling thread wait until that worker terminates. Once run() returns, the worker is TERMINATED.

4. start() versus run()

Calling start() asks the JVM to create/schedule a separate execution path and then invokes run() on that worker thread. Calling run() directly is an ordinary method call on the current thread. The program prints the current thread name in both cases: the direct run() call reports main, while the started thread reports Worker-Started.

5. sleep(), join() and isAlive()

sleep(milliseconds) pauses the current thread for a timed interval and does not release locks it owns. join() makes the calling thread wait for the target thread to finish. isAlive() is true after a thread has started and before it has terminated; it is false after completion. The program checks isAlive() shortly after start() and again after join().

6. Why join() is required for the final total

The main thread starts all four stations and then calls join() for each. It cannot print the final total until every join() returns. Each station adds its batch count to an AtomicInteger, which makes the shared update atomic. The expected total is 4 × 10 = 40 scripts for each implementation.

7. Behavioural experiments and scheduler

The program records ten completion-order trials. The order can vary because the operating-system/JVM scheduler, timing, CPU load and sleep durations affect which worker completes first. Thread priority is only a hint to the scheduler; it does not guarantee a completion order and may have little visible effect on some platforms. The program includes a priority comparison, but the actual observed order must be recorded from the student's machine.

8. Daemon thread experiment

The daemon station is marked with setDaemon(true) before start(). Main does not join it. A JVM exits when no non-daemon threads remain, even if daemon threads are still running. Therefore the daemon's completion message may not appear. Daemon threads are appropriate for background services that should not keep the application alive, not for essential work that must finish.

9. Joint conclusion

The same multithreaded program can produce different output orders because thread scheduling and execution timing are nondeterministic. Correctness must depend on synchronization and completion coordination, not on which station prints first. Here AtomicInteger protects the shared total and join() ensures all work is complete before the total is displayed.

10. ExecutorService alternative and complexity

A fixed thread pool can execute four Callable station tasks. submit() returns a Future for each task; calling get() retrieves the completed result and waits if necessary. The pool is shut down after all results are collected. This manages a bounded number p of reusable worker threads instead of manually creating a thread for every task.

For n scripts distributed evenly across p workers, idealized work time is approximately O(n/p) batches, plus scheduling and coordination overhead; total work remains O(n). With four station tasks and p=4, each station can run concurrently. With a pool smaller than four, tasks queue and execute in multiple waves. Thread stack memory is approximately O(p × S), where S is the stack size per worker, for a fixed pool. Creating one thread per station uses O(4 × S) stack memory here; for n individual scripts, one-thread-per-script could require O(n × S), while a fixed pool remains O(p × S).

11. Evidence to attach

Capture the two implementation outputs, ten observed completion-order runs, priority experiment results, the run-versus-start thread-name evidence, and the daemon behaviour. Do not present one machine's sample output as a universal scheduler guarantee. Fill the supplied experiment-log-template.csv with actual observations.

12. Conclusion

Both Thread and Runnable can execute the four scanner stations concurrently. Runnable offers better separation of task and execution mechanism, while join() and atomic shared-state updates make the final result reliable regardless of completion order. A fixed ExecutorService pool is preferable when the number of tasks grows and thread resources need to be bounded.

This report intentionally contains no student-specific details. Add screenshots and actual experiment measurements before submission.